Fine-tuning a Transformer for Patent Phrase Matching

Lab report · U.S. Patent Phrase to Phrase Matching (Kaggle late submission) · Kaggle notebook: *anan999999/patent-phrase-matching-lab* (versions 3 and 4)

Setup

All results come from one Kaggle notebook run on a T4 GPU with internet off (transformers 5.16). The notebook builds on the course starter notebook: a **deberta-v3-small** cross-encoder (He et al., 2021) with input *[CLS] anchor [SEP] target; CPC section name [SEP]*, a single-output regression head trained with MSE, 3 epochs, LR 2e-5, batch size 32, 10% warm-up. The validation split is the starter's *GroupShuffleSplit* on anchors (SEED = 42, VAL_FRACTION = 0.2): 28,914 training rows and 7,559 validation rows from 147 held-out anchors. It was never changed. The metric is the Pearson correlation *r*.

Task 1 – Fine-tuned baseline

The unchanged starter configuration reached **validation r = 0.8170** (0.7985 / 0.8136 / 0.8170 after epochs 1/2/3) in **5.6 min** of training. **Why whole anchors are held out:** each of the 733 anchors appears with about 50 targets. A random row split would put almost every validation anchor in training too, so the model could score well by memorising how each anchor is typically rated rather than by judging similarity. The validation r would be inflated, would overestimate performance on the test set (whose anchors are unseen), and would favour settings that memorise better.

Task 2 – Baseline without fine-tuning

The pretrained sentence-embedding model **all-MiniLM-L6-v2** (Reimers & Gurevych, 2019) encoded anchor and target separately (mean pooling, L2-normalised). Their cosine similarity was used as the prediction, with no training. On the same split it reached **r = 0.5814**, against **0.8170** for the fine-tuned cross-encoder.

A **bi-encoder** maps each phrase to one vector on its own, and the phrases only meet in the final cosine. This is fast and the vectors can be precomputed, but the model cannot align individual tokens across the pair or use the CPC context, and its notion of similarity is the generic one learned in pre-training. A **cross-encoder** reads both phrases (and the context) as one sequence, so self-attention can compare them token by token, and a regression head learns the raters' 0–1 scale. The +0.24 gap shows that pre-training alone already captures much of the relatedness (r = 0.58). Fine-tuning a cross-encoder on about 29k labelled pairs adds the patent-specific notion of similarity and the label scale. The gap combines two changes (architecture and training), so it is not attributable to fine-tuning alone.

Task 3 – Controlled experiments

Each run changes one factor from the baseline. A rerun of the baseline with training seed 43 (same split) measures noise: the two seeds differ by only 0.0003, so the brief's 0.005 threshold is used. Differences smaller than 0.005 are treated as no difference.

Setting changed (one factor)	Val r	Train time	Interpretation
Baseline (starter: section name, LR 2e-5)	0.8170	5.6 min	Reference
Baseline rerun, seed 43	0.8173	5.6 min	Seed noise ≈ 0.0003
A1. Context: none	0.8144	5.2 min	–0.0026: within noise
A2. Context: raw CPC code (e.g. G02)	0.8139	5.5 min	–0.0031: within noise
B1. Learning rate 1e-5	0.8000	5.6 min	–0.0170: worse, under-trained in 3 epochs
B2. Learning rate 5e-5	0.8212	5.6 min	+0.0042: best run, but within noise
Task 2: bi-encoder, no training (ref.)	0.5814	1.7 s*	Pretrained embeddings only

Table 1. Validation Pearson on the fixed anchor-grouped split (T4 GPU). *Encoding time only. None of the changes affects model size or batch size, so training time is similar across runs.

Context representation. Neither removing the context nor replacing the section name with the raw CPC code changed r beyond noise, so on this split we cannot show that the context representation matters. This is not because context is constant: validation anchors occur with 2.6 CPC classes on average, and 54% span more than one section. A plausible reason is that for most pairs the phrases themselves (e.g. *hydraulic cylinder*, *naval gun*) already signal the field, so the context rarely changes the rating. This does not show that context is useless for ambiguous phrases (the brief's *mouse* example), only that such cases are too rare here to move the overall r. **Learning rate.** 1e-5 is clearly worse (–0.017, and the lowest epoch-1 score, 0.772): with 3 epochs the model does not converge. 5e-5 gave the highest r (0.8212), but the +0.0042 gain is below the noise threshold, so it is not claimed to be better than 2e-5 without more seeds.

Task 4 – Error analysis (final model, LR 5e-5)

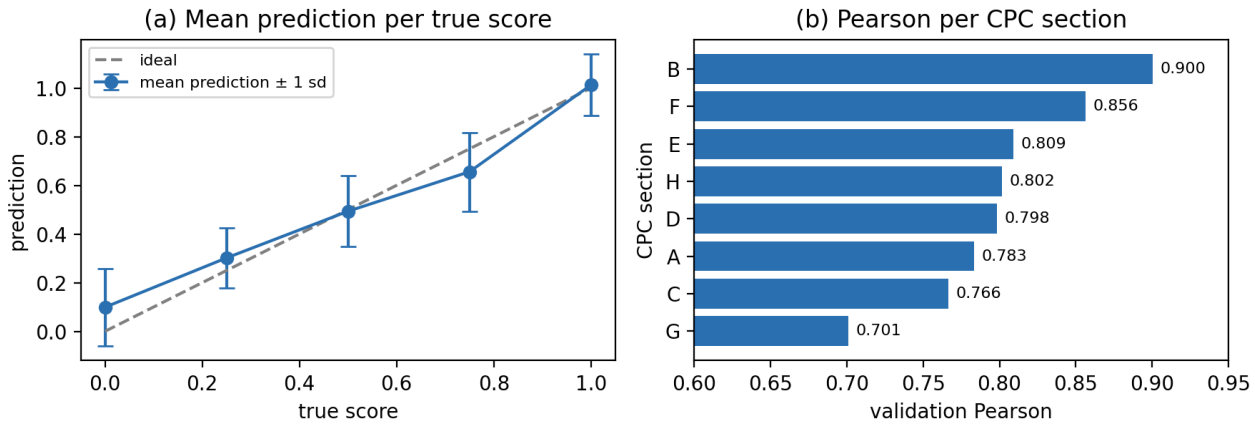


Figure 1. (a) Mean validation prediction (±1 sd) for each true score; dashed line = ideal. (b) Validation Pearson per CPC section.

Per section. The hardest sections are **G physics (0.701)**, C chemistry (0.766) and A human necessities (0.783). The easiest are B performing operations/transporting (0.900) and F mechanical engineering (0.856). In G (optics, computing, measurement), everyday words carry technical meanings (*imaging axis*, *identity*, *position*), and five of the ten largest errors are G pairs. C and A need chemical and biomedical knowledge. B and F phrases name concrete parts, where word overlap is a more reliable signal. D (12 anchors) and E (14 anchors) have few held-out anchors, so their r is noisy.

Per true score. Mean predictions are 0.099 / 0.302 / 0.493 / 0.656 / 1.012 for true scores 0 / 0.25 / 0.5 / 0.75 / 1. The middle levels are well calibrated, but the model compresses partly towards the middle: true 0 is predicted slightly high and **true 0.75 clearly low (0.656)**, so 0.75 pairs overlap with 0.5 pairs. This matches the imbalance: 0.25 and 0.5 make up 65% of training labels and 0.75 only 11%, so MSE pulls predictions towards the frequent middle levels. Score 1.0 (3%) is not compressed, because those pairs are mostly exact or near-exact matches that are easy to recognise.

Anchor	Target	CPC	True	Pred	Type
punch face	face punch	B05	0.00	0.898	1
imaging axis	imaging of axis	G02	0.00	0.835	1, 3
boom hydraulic cylinder	boom a hydraulic cylinder	E02	0.00	0.820	1, 3
imaging axis	image of axis	G02	0.00	0.771	1, 3
end pins	wood pin	G10	0.75	–0.013	2, 3
breech mechanism	breech loading naval gun	B29	0.75	0.033	2
whose identity	customer id	G06	0.75	0.060	2
activating position	active region	G06	0.00	0.676	1
secure to guard	security guard	A01	0.00	0.667	1
return structure	return to structure	D01	0.00	0.659	1

Table 2. The 10 validation pairs with the largest absolute error. Type numbers refer to the groups below.

(1) Same words, different meaning (7/10, over-prediction). The model relies on word overlap and misses that word order or a small function word changes the meaning: a *punch face* is part of a tool, a *face punch* is a blow; *secure to guard* vs *security guard*. **(2) Related without shared words (3/10, under-prediction).** *breech mechanism* → *breech loading naval gun*, *whose identity* → *customer id*: recognising these needs domain knowledge or an abbreviation (*id*). **(3) Debatable labels.** *boom hydraulic cylinder* vs *boom a hydraulic cylinder* and *imaging axis* vs *imaging of axis* are near-identical strings rated 0, and *end pins* → *wood pin* rated 0.75 is surprising. Part of these errors is likely rater noise rather than model failure.

Task 5 – Late submission and leaderboard scores

The final model was chosen by **validation score only**: the highest-r run in Table 1 (LR 5e-5), selected automatically inside the notebook. Two late submissions were made. Both came from identical code, one via the Kaggle CLI and one re-submitted from the website, and both are reported:

Submission	Notebook version	Final model	Validation r	Public r	Private r
57807770	3	deberta-v3-small, LR 5e-5	0.8212	0.7961	0.8157
57808203	4 (same code)	deberta-v3-small, LR 5e-5	0.8212	0.7961	0.8157

Table 3. All submissions. A screenshot of the Submissions tab is attached separately.

Validation (0.8212) and private (0.8157) agree closely, a gap of 0.0055. This suggests the anchor-grouped split estimates performance on unseen anchors well. The public score is about 0.02 lower. Reasons the three can differ: (i) **different anchors and section mix**: performance varies strongly by section (0.70–0.90, Fig. 1b), so a test part with

more G or C pairs scores lower; (ii) **test-set size**: the public part is the smaller slice of the hidden test set, so its r has more sampling noise, and the larger private part is the more reliable estimate; (iii) **selection on validation**: the final model was the best of five settings on the same validation set, so its validation r is slightly optimistic (its margin over the baseline was within noise); (iv) the model is trained on only 80% of *train.csv*.

References

Aslanyan, G., & Wetherbee, I. (2022). Patents Phrase to Phrase Semantic Matching Dataset. arXiv:2208.01171.

He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. arXiv:1908.10084.

Kaggle. U.S. Patent Phrase to Phrase Matching competition (Overview, Data, Rules, Code Requirements). https://www.kaggle.com/competitions/us-patent-phrase-to-phrase-matching

Google Research (2022). Announcing the Patent Phrase Similarity Dataset.

Course materials: *patent_phrase_starter.ipynb* (pipeline, split and training code reused and wrapped in a function) and *download_weights.ipynb* (offline checkpoints), github.com/bk-anupam/generative_ai_shared.

Hugging Face Transformers and Sentence Transformers documentation. Models: microsoft/deberta-v3-small (MIT), sentence-transformers/all-MiniLM-L6-v2 (Apache-2.0). No public competition notebooks or solution write-ups were used.